

The Minesweeper Project

This project will guide you to build an automated Minesweeper simulator using object-oriented programming and custom data structures to generate a massive dataset of matches.

Subsequently, you will step into a data scientist role, utilizing the Java Streams API and Collections to query this dataset, analyzing bots' win rates, and extract the fastest victories.

Milestone 1: The Intelligent Board & Encapsulated Algorithms

Objective: Build the core in-memory representation of the Minesweeper game. This milestone fuses clean Object-Oriented design with fundamental algorithmic logic, requiring you to combine matrix traversal with custom, fully encapsulated data structures.

Tasks:

1. Cell Representation (OOP Basics):

- Create a **CellState** enumeration representing the visual states: HIDDEN, REVEALED, and FLAGGED.
- Create an enumeration *GameOutcome*. The *GameOutcome* can be one of *IN_PROGRESS*, *VICTORY*, *DEFEAT*.
- Create a **Cell** data type containing properties for whether it is a mine (boolean), its current CellState, and the number of adjacent mines (int).

2. Encapsulated Custom Queue (Data Structures):

- **Queue implementation.** Implement a **CoordinateQueue** class that provides a FIFO structure for storing grid coordinates during gameplay. The implementation must be self-contained: use of any class from the **java.util** package, as well as **native arrays**, is strictly **prohibited**.
- **Encapsulation requirement.** To prevent abstraction leaks, implement the queue's nodes as a private static class **CoordinateNode** *inside* the **CoordinateQueue** class. The outside world must remain completely unaware of how the queue operates internally.

3. The Board Class (Matrix Logic & Custom BFS):

- Implement a **Board** class that represents the Minesweeper grid. The board internally stores a 2D array of **Cell** objects.
- Implement a **constructor** that takes the grid dimension and the number of mines as input. It must initialize the 2D grid, place the requested number of mines at random positions, and then compute the adjacent-mine counts for every cell. Validate inputs: size must be positive, and numMines must be non-negative and strictly less than size * size. If some arguments do not satisfy these requirements, throw an **IllegalArgumentException**.

- Implement a **private calculateAdjacentMines()** method. You must traverse the grid and count the **neighboring mines** for each safe cell. *Hint: Use displacement arrays (dx, dy) to evaluate the 8 neighbors cleanly without deeply nested boundary checks.*
- Implement the **revealCell(int row, int col)** method. Reveal the cell at the given coordinates, transitioning its state from HIDDEN to REVEALED. If the revealed cell has zero adjacent mines, the method must automatically uncover all reachable safe neighbors (the cascade effect). The cascade must be performed iteratively using your CoordinateQueue as a Breadth-First Search algorithm. Recursion is not allowed.
- Implement the method **getGameState()**. Method returns the current GameOutcome:
 - DEFEAT — if any cell containing a mine has been revealed.
 - VICTORY — if every non-mine cell has been revealed.
 - IN_PROGRESS — otherwise.

Milestone 2: The Automated Player & Telemetry (Interfaces & Linked Lists)

Objective: Design an automated simulation system where AI "Bots" play the game. You must build this system from the bottom up: starting with the custom data structures, moving to the interfaces, and finally implementing the bot logic.

Tasks:

1. The Telemetry Data Structure (Data Structures):

Before a bot can play, it needs a place to record its memory.

- **1.1 Create the Move data type:** This simple class should contain the row (int), column (int), and a wasSafe (boolean) flag indicating if the move triggered a mine.
- **1.2 Create the NodeMove data type:** This is the fundamental building block of a singly linked list, storing the actual Move data payload. It also contains a next pointer linking to the subsequent node, enabling sequential traversal of the bot's move history.
- **1.3 Create the MyLinkedList data type:** Implement a singly linked list from scratch to store Move objects.
 - **Constraint: You are strictly forbidden from using java.util.ArrayList, LinkedList, or native arrays.** Implement an insert(Move move) method and a getHead() method.

2. The Player Class: Now that the telemetry structure exists, define the Player class.

2.1 Implement Player class. Player class holds a MyLinkedList attribute storing the history of moves made during the game. The player must be associated with a Board instance (passed via constructor) so it knows which board to play on.

- **GameOutcome playTurn():** executes a single move by randomly selecting one cell that has not yet been revealed and revealing it. The move is recorded in the player's move history (with wasSafe set according to whether the cell was a mine). The method returns the resulting GameOutcome: VICTORY or DEFEAT if the game is over, IN_PROGRESS otherwise.
- **MyLinkedList getMoveHistory():** Returns the custom linked list containing the sequence of moves made by the player.

4. The Simulation Engine & Factory

Tie everything together. You will build a Simulator class to create players, run the game loop autonomously, and extract the data from the custom linked list to generate a massive, summarized CSV dataset of the played matches (e.g., 1000 matches).

1. Create a test class **GamesSimulator** with a static method *Player createBot(Board board)*. It should return a Player object.
2. **The Mass Simulator (main method):** For each match, initialize a Board (e.g., an 8x8 grid with 6 mines to ensure a reasonable win rate).
 - You will create a csv file with the following columns:

MatchId,BotType,Result,TimeMs,TotalClicks
 - Run the game loop: The bot should playTurn() repeatedly until it either hits a mine or wins.
 - Once a single match ends, evaluate the result (VICTORY or DEFEAT) and retrieve the MyLinkedList from the player to count the TotalClicks. Append this single summary line to your CSV file before starting the next match loop.

Milestone 3: The Data Analyst & The Hall of Fame (Streams & BST)

Objective: You have successfully built a Minesweeper engine and simulated 1,000 automated matches. Now, it is time to change your role from Game Developers to Data Scientists. In this final milestone, you will read your massive matches.csv dataset, use **Java Streams** to extract statistical insights, the **Java Collections Framework** to store and group the data, and apply the **Java Streams API** to extract statistical insights.

Tasks:

1. Data Modeling & Parsing (OOP)

Before analyzing the data, you need an object representation of a single match.

- Create a **MatchSummary** data type (class or record) with properties that strictly match your CSV columns: matchId (int), botType (String), result (String), timeMs (long), and totalClicks (int).
- **Sorting Contract:** Make your MatchSummary class implement the Comparable interface. Matches are naturally sorted by their timeMs in ascending order (fastest matches first).
- Create a Factory class with a static method for parsing a match from a csv line.

2. The Telemetry Dataset (Java Sets)

- Create a **MatchDataset** data type that uses a SortedSet to store all matches in the file.
- Add a static method to the factory for parsing the whole .csv file. This method must return an object of type **MatchDataset**.
- Add the following methods to the **MatchDataset** data type to implement some sequential treatment functionality:

1. public Double getAverageClicksByResult(String result)

Description: Calculates the average number of clicks made by bot when the match ends in a concrete result (e.g., when it secures a "VICTORY"?). If it cannot be calculated (for example, if there is no matching data), the method must return 0.0.

2. public MatchSummary getMatchWithHighestClickRate()

Description: Finds the match that has the highest "Click Rate". You must define the click rate as (double) totalClicks / timeMs. *Safety note:* If timeMs is 0, treat it as 1 to avoid a division by zero error. Use Comparator.comparing along with the .max() terminal operation.

3. The Grand Finale (Execution & Testing)

In this final section, you will create the main entry point of your application to load your simulated dataset and run your functional stream queries to generate a complete telemetry report.

- Create a new class named **TestMatchesAnalysis**
- Use your factory class to read the matches.csv file.
- Execute the four analytical methods in MatchDataset. You must print the results of each query to the console in a clean, professional, and easy-to-read format.